

Two finer-grained compilation model for named modules

Document #: P3057R0
Date: 2023-11-21
Project: Programming Language C++
Audience: SG15
Reply-to: Chuanqi Xu
<chuanqi.xcq@alibaba-inc.com>

Contents

- 1 Abstract
- 2 Proposed solutions
 - 2.1 Used Files based solution
 - 2.1.1 foo and bar example
 - 2.1.2 Hello World Example
 - 2.2 Hash of declarations solution
 - 2.2.1 Example
 - 2.2.2 For the format of the output
- 3 Demo
- 4 Summary

§ 1 Abstract

One of the major concerns for adopting named modules is that named modules will bring a lot dependencies altogether. So that the incremental build within modules may compile more files if we changed one of the module interfaces or the included headers. To solve/mitigate this issue, this paper presents two new compilation models for named modules with finer-

grained dependency management. The demo tools needed in the compiler side are provided.

§2 Proposed solutions

We propose 2 potential solutions in this paper. One is based on the files used during the compilation. Another is based on the hash values of declarations used during the compilation.

We can find tools and the command line options used in this section in [Demo](#) section.

§2.1 Used Files based solution

For a specific source file, we can collect used files from modules during the compilation of the source file. Then we can try to rebuild the source file if all the recorded used files doesn't change.

§2.1.1 foo and bar example

```
//--- foo.h
inline int foo() {
    return 43;
}

//--- bar.h
inline int bar() {
    return 43;
}

//--- foo.cppm
module;
#include "foo.h"
#include "bar.h"
export module foo;
export using ::foo;
export using ::bar;

//--- use.cpp
import foo;
int use() {
    return bar();
}
```

We can compile normally use.cpp and get the used files from modules.

```
$ clang++ -std=c++20 use.cpp -c -o use.o -fplugin=  
    <path/to>/ClangGetUsedFilesFromModulesPlugin.so -  
    fplugin-arg-get_used_files_from_modules-  
    output=used_files -fmodule-file=foo=foo.pcm  
  
$ cat used_files  
foo.cppm  
bar.h
```

We can find that foo.h is not included in the list. Since the declarations in foo.h doesn't contribute to the compilation process of use.cpp.

So the build system can use the information to avoid recompiling use.cpp in case only foo.h changes.

Note that here the term change means the change in the file systems instead of in the traditional dependency analysis. Since foo.cppm includes foo.h and foo.cppm will always change if foo.h changes in the traditional dependency analysis.

§ 2.1.2 Hello World Example

```
// Hello.cppm  
module;  
#include <iostream>  
export module Hello;  
  
class Hello {  
public:  
    void hello() {  
        std::cout << "Hello World" << "\n";  
        std::cout << "Hello " << "\n";  
    }  
};  
  
export void hello() {  
    Hello h;  
    h.hello();  
    h.hello();  
}  
  
// Use.cpp  
import Hello;  
int main() {
```

```
    hello();  
}
```

Let's get the used files for Use.cpp by:

```
$ clang++ -std=c++20 Use.cpp -c -o Use.o -fplugin=  
    <path/to>/ClangGetUsedFilesFromModulesPlugin.so -  
    fplugin-arg-get_used_files_from_modules-  
    output=used_files -fmodule-file=Hello=Hello.pcm  
  
$ cat used_files  
/usr/include/bits/types/struct_FILE.h  
/usr/include/bits/types/__FILE.h  
/usr/include/bits/types/FILE.h  
<path/to>/Hello.cppm
```

Note that <iostream> is not included in the list! The reason is the declarations in <iostream> doesn't contribute to the compilation.

We can verify this by converting both hello() and Hello::hello() to inline explicitly. It is necessary to convert Hello::hello() too since in-class member definition is no longer implicitly inline in module purview.

```
module;  
#include <iostream>  
export module Hello;  
  
export class Hello {  
public:  
    inline void hello() {  
        std::cout << "Hello World" << "\n";  
        std::cout << "Hello " << "\n";  
    }  
};  
  
export inline int hello() {  
    Hello h;  
    h.hello();  
    h.hello();  
}
```

After we converted them, the output of used files becomes to:

```
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../../include/c++/9/bits/uniform_int_dist.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../../include/c++/9/limits
```

```
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/basic_ios.tcc  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/basic_string.tcc  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/stl_pair.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/cstdint  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/stl_heap.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/nested_exception.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/ostream.tcc  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/exception.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/move.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/exception  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/backward/binders.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/cstdlib  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/streambuf  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/type_traits  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/stl_algo.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/ios_base.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/string_view.tcc  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/range_access.h  
<path/to>/Hello.cppm  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/streambuf.tcc  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/ostream  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/x86_64-redhat-  
    linux/bits/c++config.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/iosfwd  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/ostream_insert.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/stringfwd.h
```

```
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/memoryfwd.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/iostream  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/cxxabi_init_exception.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/basic_ios.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/localefwd.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/x86_64-redhat-  
    linux/bits/c++locale.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/stl_iterator_base_types.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/char_traits.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/stl_algobase.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/postypes.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/cwchar  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/exception_ptr.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/functexcept.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/cpp_type_traits.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/basic_string.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/stl_function.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/stl_iterator_base_funcs.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/typeinfo  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/string_view  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/hash_bytes.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/stl_tempbuf.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/clocale  
/usr/include/bits/types/struct_FILE.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/x86_64-redhat-  
    linux/bits/ctype_base.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/alloc_traits.h
```

```
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/functional_hash.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/locale_facets.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/stl_construct.h  
/usr/include/bits/types/__FILE.h  
/usr/include/bits/types/FILE.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/locale_classes.tcc  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/istream  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/streambuf_iterator.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/new  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/ptr_traits.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/cctype  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/stl_iterator.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/debug/debug.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/std_abs.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/x86_64-redhat-  
    linux/bits/cctype_inline.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/cwctype  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/stdexcept  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/cstdio  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/istream.tcc  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/system_error  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/x86_64-redhat-  
    linux/bits/error_constants.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/locale_facets.tcc  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/algorithmfwd.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/initializer_list  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/locale_classes.h
```

```
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/string  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/bits/allocator.h  
/usr/lib/gcc/x86_64-redhat-  
    linux/9/../../../../include/c++/9/x86_64-redhat-  
    linux/bits/c++allocator.h
```

This feature is pretty helpful if the header is a first-party header. Then we can avoid compiling `Use.cpp` if only the header changes.

Sadly, we still need to recompile `Use.cpp` if the implementation of `Hello::hello()` changes or the class `Hello` adds new members. Although we can workaround the issue by putting class `Hello` to a partition or even another plain text and include it here, it is not so convenient.

Luckily, it is possible solve the issue by another solution presented here. The hash of declarations based solution.

§ 2.2 Hash of declarations solution

During the compilation of a specific file, we can record the declarations used in modules. Then when the module file updates, we can query and compare these declarations. Then we can skip recompiling the file if no used declarations changes.

I heard the idea from Mathias Stearn for the first time.

§ 2.2.1 Example

```
///a.cppm  
export module a;  
  
export int a() {  
    return 43;  
}  
  
namespace nn {  
    export int a(int x, int y) {  
        return x + y;  
    }  
}  
  
export template <class T>  
class Templ {
```



```

public:
    T get() { return T(43); }
};

export class C {
public:
    void member_function() {}
};

// b.cpp
#include <iostream>
import a;

int main() {
    std::cout << a() << std::endl;
    Templ<int> t;
    std::cout << t.get() << std::endl;
}

```

When compiling b.cpp, we can query the used decls during the compilation model by another plugin ClangGetDeclsInModulesPlugin:

```

$clang++ -std=c++20 b.cpp -fmodule-file=a=a.pcm -c -o b.o -
    fplugin=<path/to>/ClangGetDeclsInModulesPlugin.so -
    fplugin-arg-decls_query_from_modules-output=b.json

```

```

$ cat b.json

```

```

[
  {
    "decls": [
      {
        "a": {
          "Hash": 3379170117,
          "col": 12,
          "kind": "Function",
          "line": 3,
          "source File Name": "/home/chuanqi.xcq/llvm-project-
            for-work/build/ModulesDatabase/a.cppm"
        }
      },
      {
        "Templ": {
          "Hash": 222160723,
          "col": 7,
          "kind": "ClassTemplate",
          "line": 26,

```

```

        "source File Name": "/home/chuanqi.xcq/llvm-project-
        for-work/build/ModulesDatabase/a.cppm"
    },
    {
        "Templ::get": {
            "Hash": 3343854614,
            "col": 7,
            "kind": "CXXMethod",
            "line": 28,
            "source File Name": "/home/chuanqi.xcq/llvm-project-
            for-work/build/ModulesDatabase/a.cppm"
        }
    },
    ],
    "module": "a"
}
]

```

Then when a.pcm changes and we're trying to recompile b.cpp, we can query these declarations in a.pcm by:

```

$clang-named-modules-querier a.pcm -- a nn::a Templ::get
[
  {
    "a": {
      "Hash": 3379170117,
      "col": 12,
      "kind": "Function",
      "line": 3,
      "source File Name": "<path/to>/a.cppm"
    }
  },
  {
    "nn::a": {
      "Hash": 1071306246,
      "col": 16,
      "kind": "Function",
      "line": 20,
      "source File Name": "<path/to>/a.cppm"
    }
  },
  {
    "Templ::get": {
      "Hash": 3343854614,
      "col": 7,
      "kind": "CXXMethod",
      "line": 28,

```

```
    "source File Name": "<path/to>/a.cppm"  
  }  
}  
]
```

(unqualified name are treated as if they are under global namespace)

We can find that all these used declarations are not changed. So we're allowed to skip the compilation for `b.cpp`.

§ 2.2.2 For the format of the output

The format of the informations of declarations is not decided yet. This is still an open area and we can try to experiment and discuss.

Further more, maybe we're possible to define DSL for programmers/build systems/static analyzers to query the informations they want. It implies that we may treat the BMIs as databases!

§ 3 Demo

We can find the demo to help to implement these strategies in <https://github.com/llvm/llvm-project/pull/72956>. These implementations are not polished. We can push it further if people are interested. Also it will be highly appreciated if build systems vendors are willing to make experiments with them.

§ 4 Summary

This paper presents 2 more finer-grained compilation model for named modules. To allow the build systems/users to understand what is used in the module file during the compilation. So that we may skip some unnecessary recompilations.

Both of the compilation model have a demo implementation in the compiler side.

The hash value based solution looks pretty appealing while it is hard to implement both for build systems and compilers (I believe the current implementation is not production ready). Also the cost of computing

hashes and comparing a lot of declarations is a concern too. But it should still worth to give it a try.

The used files solution should be much more stable and easier to implement. Maybe we can choose this solution as a first step to optimize the compilation of modules.